```
    function zoomed() {
      var currentRotate = projection.rotate()[0];
      projection.scale(mapZoom.scale());
      d3.selectAll("path.graticule").attr("d", geoPath);
      d3.selectAll("path.countries").attr("d", geoPath);

      d3.selectAll("circle.cities")
      .attr("cx", function(d) {return projection([d.y,d.x])[0]})
      .attr("cy", function(d) {return projection([d.y,d.x])[1]})
      .style("display", function(d) {return parseInt(d.y) + currentRotate <
     90 && parseInt(d.y) + currentRotate > -90 ? "block" : "none"})
    }

…code to add manual zoom and zoom buttons…

…code to draw graticule, countries and cities…

…code to create and clear center and bounding box…
  }
```

A plugin by Jason Davies known as `d3.geo.zoom` (https://www.jasondavies.com/maps/rotate/) abstracts this functionality.

But this map still has the problem of a graphical artifact from the graticule outline, which must be removed when drawing globes. Another problem is seeing through the globe to the other side. This might be a fine idea, if it didn't also muddle the SVG drawing code so that the shapes are drawn poorly when they get near the border (notice how poorly Antarctica looks in figure 7.12). Also, our cities are drawn above the paths, even when they're ostensibly on the other side of the world (for example, Karachi).

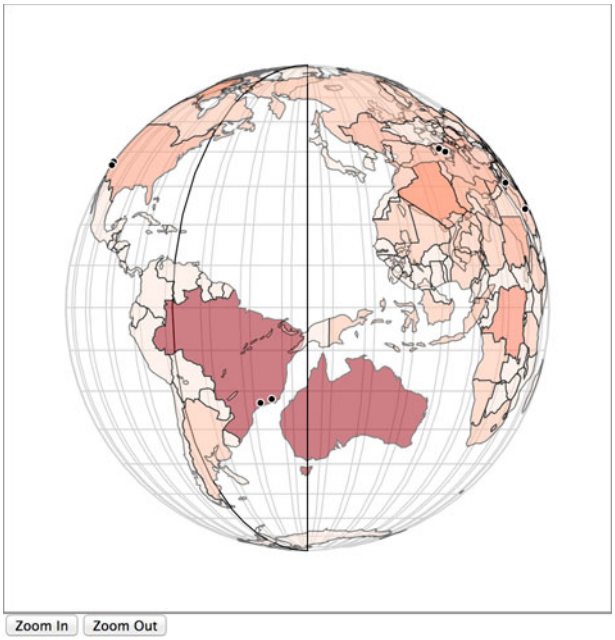


Figure 7.12 A globe with a transparent surface. You can see Australia through the globe because the projection doesn't by default clip this. Cities are drawn at the correct coordinates but are uniformly drawn above the features because the `<circle>` elements are drawn on top of the `<path>` elements in the DOM.